Frame AI

Complete Usage Guide

Every feature, every option, every button: what it does and how to use it.

This is not a setup guide. It is a **usage** guide: the frontend screens, the sheet columns, the options for making videos, how the backend behaves, and common problems.

Source: Frame-Ai-main.zip (frontend: Videos.jsx, config.js · backend: Python pipeline, drive_runner.py, GitHub Actions workflow, README).

Contents

Contents 2

0. How to read this guide 5

1. What Frame AI is 6

1.1 A video's journey (bird's-eye view) 6

1.2 Folder map in Drive 6

2. Core concepts 7

2.1 Playlist (Project) and Singles 7

2.2 The three settings levels (the most important rule) 7

2.3 Status: the state of a video 7

2.4 Input Type: what a video is made from 8

3. Control room: global settings 9

3.1 Keys 9

3.2 Models (priority order) 9

3.3 Rate limits 10

3.4 Render machine 10

3.5 Connect backend 10

4. Playlist (Project) settings 11

4.1 Content and shape 11

4.2 Engine, research and scenes 11

4.3 Narrator: the voice 12

4.4 Default thumbnail template 12

5. The Videos tab: daily work 13

5.1 The stats line at the top 13

5.2 Adding a new video 13

5.3 Sheet upload: many videos at once 14

5.4 The video card: every detail 14

5.5 The three buttons above the list 16

6. Input types: Prompt, Script, Source 17

6.1 Prompt: Frame writes the script 17

6.2 Script: your script exactly as written 17

6.3 Source: adapting raw text 17

7. The playlist sheet columns 19

8. The look of the video: engine, shape, quality, scenes . . . 21

8.1 Video engine: Manim or Remotion . . . 21

8.2 Aspect ratio (shape of the video) . . . 21

8.3 Render quality . . . 21

8.4 Scene pace: seconds per scene and scene count . . . 22

8.5 Language and Urdu notes . . . 22

9. Scenes: your own storyboard or code file . . . 23

9.1 Two kinds of scene . . . 23

9.2 Writing scenes as plain text . . . 23

9.3 Scene count: Frame handles it for you . . . 23

9.4 Which scene goes where . . . 24

9.5 If your own file cannot run . . . 24

9.6 Scene report . . . 24

10. Custom Clips: your own image/video overlays . . . 25

11. Thumbnails . . . 26

Thumbnail studio settings (project level) . . . 26

12. Singles: stand-alone videos . . . 27

13. Production tab: whose turn is it . . . 28

13.1 Plan options . . . 28

13.2 What the plan produces . . . 28

14. Backend behaviour: what you will see . . . 29

14.1 When it runs . . . 29

14.2 What happens to one video . . . 29

14.3 Outcomes: Completed, Failed or back to Waiting . . . 30

14.4 Free-tier considerations . . . 30

15. Extra Config (JSON): every option . . . 31

Examples . . . 32

16. Common problems and fixes . . . 33

17. Quick recipes: by task . . . 35

17.1 Your first video from a prompt . . . 35

17.2 A video from your own script . . . 35

17.3 A video from a book or article (Source) . . . 35

17.4 Many videos at once . . . 35

17.5 Making a vertical Short / Reel 35
17.6 Urdu narration with English screen text 35
17.7 Rebuilding a video 35
17.8 Using your own scene (Manim or Remotion file) 36
17.9 Keeping the voice consistent 36

Appendix A: Gemini voices 37

Appendix B: Short glossary 38

0. How to read this guide

The guide is ordered so you first learn the basic concepts (sections 1-2), then the settings (3-4), then the daily work of adding and running videos (5-8), then advanced options (9-13), and finally the backend, reference tables and troubleshooting (14-17).

New user? Read sections 1, 2 and 5, then jump to the “Quick recipes” in section 17.

Want the full list of options? Section 4 (project settings), section 7 (sheet columns) and section 15 (Extra Config JSON) hold every field in table form.

> **Important clarification**
> The zip contained only two frontend files: **Videos.jsx** (the Videos tab) and **config.js** (the list of all settings, defaults and merge rules). The code for the Control room, Singles, Production tab, Thumbnail studio and Scenes section was not in the zip. Their options are described from **config.js** and the **README**, so button labels or layout may differ slightly. Wherever this applies the text says “(screen code not in the zip)”.

> **Names**
> Status and input names such as **Hold**, **Waiting**, **Prompt**, **Script** appear exactly like this on screen, so they are kept as-is.

1. What Frame AI is

Frame AI turns a **topic** (or your own **script**, or the **text** of a book or article) into a complete educational video: narration, animated scenes, captions (.srt) and a final MP4. English and Urdu are supported.

The easiest way to understand it is that it has **three parts**:

Part	What it is	Your job
Frontend (web app)	The screen where you log in, fill in settings and create playlists and videos. It has no server or database of its own.	Write or add videos, change settings, review thumbnails, watch statuses.
Google Drive	All your data lives in your Drive inside the **Frame AI** folder: Sheets (the video list), JSON config files, and the finished videos.	You can also edit the sheet directly if you want.
Backend (GitHub Actions)	Runs by itself every 6 hours (or when you press “Run workflow”), reads the config and **Waiting** videos from Drive, builds each video and saves it to the Output folder.	Nothing. Set videos to **Waiting** and wait.

1.1 A video's journey (bird's-eye view)

#	What happens	Status
1	You add a video to a playlist (prompt, script, or sheet upload).	Hold or Waiting
2	You set the video to **Waiting** (release it).	Waiting
3	On its next run the backend picks it up, merges the Drive config and starts building.	In Process
4	Web research, script, voice, scene plan, rendering each scene's animation, then joining everything.	In Process
5	The video is saved as **<Video Title>.mp4** in the Output folder, with **.srt** captions and a scene report.	Completed
6	If something breaks, the reason is written in the sheet's **Notes** column.	Failed

1.2 Folder map in Drive

```
Frame AI/
 ├─ Application Config/
 │    ├─ frame_config.json        <- global settings from the Control room (keys, models, limits)
 │    ├─ Projects (sheet)         <- folder id and sheet id of every playlist
 │    └─ Production Plan (sheet)  <- which playlist is active, videos per turn
 ├─ <Playlist name>/
 │    ├─ project_config.json      <- this playlist's settings (narrator, language, look)
 │    ├─ <Playlist name> - Record (sheet)
 │    │      ├─ Playlist tab      <- each row = one video
 │    │      └─ Log tab           <- backend history (started / saved / failed)
 │    ├─ Output/                  <- finished videos (if a video has no output folder of its own)
 │    └─ <each video's own folder>/ Input/ (clips/ ...) and Output/
 └─ Singles/                      <- stand-alone videos (own sheet: "Singles - Record")
```

2. Core concepts

2.1 Playlist (Project) and Singles

A **Playlist** (called **Project** in the frontend) is a group of videos with its own sheet, folder and settings: narrator voice, language, shape, audience, default description, default thumbnail template and so on. A playlist's settings apply to all its videos unless a video gives its own value.

Singles are for stand-alone videos. They work like a playlist, except: own sheet (**Singles - Record**), own narrator/thumbnail style, **no shared description** (only the video's own), and the video goes to **Singles/Output/**. Details in section 12.

2.2 The three settings levels (the most important rule)

Every setting can come from three places. The level **closest to the video** wins:

Level	Saved where	What it controls	Priority
GLOBAL (Control room)	Frame AI/Application Config/frame_config.json	Gemini keys, model lists, rate limits, render-machine settings. Applies to all playlists.	Lowest
PROJECT (playlist)	<playlist folder>/project_config.json	Narrator, language, shape (aspect), audience, default minutes, engine, quality, scene settings, thumbnail template.	Middle
VIDEO (row)	One row of the playlist sheet	Topic, minutes and optional overrides (language, voice, aspect, engine, etc.).	High
Extra Config (JSON)	The row's **Extra Config (JSON)** column	Any VideoConfig field (section 15). Overrides even the row's other columns.	Highest

> **Tip**
> Example: the playlist language is **en** but one row has Language = **ur**, so only that video is made in Urdu. If that row's Extra Config also says **{"language": "en"}**, the Extra Config wins.

An empty value means “take it from the level above”. That is why dropdowns in the video card start with **Playlist default** or **Project default**.

2.3 Status: the state of a video

Status	Who sets it	Meaning
Hold	You	You are not ready. The backend **never touches** it. This is the default for new videos (an empty status is also treated as Hold).
Waiting	You	You released it. The backend may build it on its next run.
In Process	Backend	Being built right now. Notes shows **[gh-action] started ...**.
Completed	Backend	Built. The **Output Link** (and captions link) is filled in the sheet.
Failed	Backend	Could not be built. The reason is in **Notes**.
Uploaded	Backend (not set by this backend today)	Reserved for YouTube upload. The backend in this zip does not upload.

You can only switch between **Hold** and **Waiting**. Once a video is In Process / Completed / Failed, treat the status as the backend's. To rebuild a video use the card's **Set back to Waiting** button (section 5.4).

> **Remember**
> If a backend run is cancelled midway and a video stays **In Process** (with [gh-action] in Notes), the next run picks it up again by itself. You do not need to do anything.

2.4 Input Type: what a video is made from

Input Type	What you provide	What Frame does
Prompt	Only a topic / prompt (and an optional custom instruction).	Does web research, **writes the script itself**, then builds the video.
Script	Your finished script.	Uses the script **exactly as written** as narration. It is only cleaned and split into scenes. Words are not changed.
Source	Raw text (book, article, notes).	Adapts the text according to the **Source Mode** you choose (polish / rewrite / reference / custom).

All three are explained in full in section 6.

3. Control room: global settings

The Control room holds settings common to **all playlists**. They are saved to **frame_config.json**. After any change you must **Save**, otherwise the backend will not see the new values. (The Control room screen code was not in the zip; options come from config.js.)

3.1 Keys

Option	What to enter	Notes
Gemini API keys	One key per line.	Used for scripts, plans and code. **At least one key is required**, otherwise the backend stops (“keys.gemini is empty”). Several keys multiply your free quota.
Gemini TTS keys	Optional. One per line.	Separate keys for voice (text-to-speech). **If empty, the Gemini keys are reused.**
Cloud TTS keys	Optional.	Google Cloud Text-to-Speech as a **fallback engine** when Gemini cannot make the voice.

> **Heads up**
> Google's rate limits apply per **project**, not per key. Five keys from the same Google Cloud project do not give 5x the quota. The real benefit comes when keys belong to **different Google Cloud projects**.

3.2 Models (priority order)

In every list the **first model is tried first**; if it fails because of quota or limits the next one is used. A model your project cannot use (404/403) is dropped from the list automatically, not the key. One model name per line.

List	Used for
Light models	Cheap, light JSON work: concepts, key points, short scene headings.
Medium models	The animation plan and outline.
Code models	Heavy work: script plan, narration, and writing and repairing Manim/Remotion code.
TTS models	Voice models; first = first choice. A playlist can **pin** one so the voice stays steady (section 4.3).
Transcription models	Stored for later. **Not yet passed to VideoConfig**, so no effect today.

Default chains at a glance: light starts with *gemini-3.5-flash-lite*; medium and code start with *gemini-3.8-flash* (code has *gemini-2.5-pro* second); TTS starts with *gemini-3.8-flash-tts*. On the free tier each model has its own daily limit, so a long chain increases the daily allowance.

3.3 Rate limits

Option	Default	Meaning
Text requests per minute	5	Free-tier limit. **Lite models automatically get x3.**
TTS requests per minute	4	Rate of voice requests.
Longest quota wait (seconds)	240	How long it will wait for quota to return. If the wait would be longer, the run **stops instead of sleeping** (the video goes back to Waiting and resumes next run).
Scenes per TTS call	4	How many scenes' narration is sent in one voice call. **1 turns batching off.**
Characters per TTS call	4000	Maximum total characters per voice call.
Scenes written per Gemini call	4	How many scenes' narration is written per Gemini call.

3.4 Render machine

Option	Default	Meaning
Urdu screen font	Noto Naskh Arabic	Font for Urdu text on screen. It **must be installed** on the rendering machine, otherwise Urdu text shows as boxes.
Work folder	build	Script, audio and clips while building.
Output folder	output	Finished videos. A sub-folder per playlist is added.
TTS cache folder	tts_cache	Voice cache. It survives deleting the work folder, so voices need not be regenerated.

> **Tip**
> On the GitHub Actions backend the work folder is always **build** and output is always **output** (the backend forces them). Changing these only affects local/manual runs.

3.5 Connect backend

Connect backend in the Control room gives a token (**Copy token**) that lets the backend reach your Google Drive. If the backend reports “Google refused the refresh token”, get a new token here and replace the old one. This is a one-time connection; you do not need it in daily use.

4. Playlist (Project) settings

Every playlist has its own settings. The **defaults** below are what a new project starts with. A video's row can override them.

4.1 Content and shape

Option	Default	Values / notes
Narration language	en	**en** English or **ur** Urdu. Script, voice and audio use this language.
On-screen text language	Same as narration	Language of text shown on screen. You can narrate in Urdu with English screen text (or the reverse).
Default video shape (aspect ratio)	16:9	16:9 Wide (YouTube) · 9:16 Vertical (Shorts, Reels, TikTok) · 1:1 Square · 4:5 Portrait (feed posts). Section 8.2.
Default description	(empty)	Added **below** every video's own description. A video with no description of its own gets just this one. (Record only; the backend does not read it when making the video.)
Audience	curious beginners	Sets the level of the script, e.g. “grade 8 students”.
Default length (minutes)	3	Used when a video row leaves Minutes empty.

4.2 Engine, research and scenes

Option	Default	Notes
Video engine	Let AI choose (auto)	auto / manim / remotion / both. Section 8.1.
Keep my engine choice fixed	Off	When on, if the chosen engine fails on a scene it does **not** switch to the other engine; the video is marked **Failed**. (No effect when engine = auto.)
Search the web about the topic before writing	On (recommended)	The backend does one Google search per video and writes from those facts. **If the search fails the video is marked Failed**; it is never written without research.
Seconds per scene	15	Bigger = fewer scenes = fewer renders (faster). Smaller = more scenes (more variety, slower).
Chapter length (minutes)	2	Only for videos longer than 4 minutes: the script is planned in chapters of about this length.
Animation steps per sentence (min)	2	Minimum animation steps per sentence.
Max scenes	60	Upper limit of scenes in a video.
Render quality	Medium	Low / Medium / High. Section 8.3.
Auto-repair attempts (Manim code / Remotion code)	2	How many times Gemini tries to fix a scene whose code fails.
Allow LaTeX	Off	For formulas in Manim. **Never with Urdu screen text** (it turns itself off).

4.3 Narrator: the voice

The narrator settings define the playlist's voice. These options are saved in the project config:

Option	Default	Notes
Voice	Kore	One of 30 Gemini prebuilt voices (full list in Appendix A). Putting a Voice in a row changes only that video.
Style (tts_style)	calm, clear and friendly, like a good teacher	The delivery style. 6 presets: calm teacher · upbeat YouTube explainer · warm bedtime storyteller · serious documentary narrator · gentle (absolute beginners) · confident news anchor. You can also write your own sentence.
TTS engine	auto	**auto** = Gemini first, Cloud TTS if that fails · **gemini** = Gemini only · **cloud** = Google Cloud TTS only.
TTS model (pin)	(empty)	Pick a model and it is tried **first**, keeping the playlist's voice consistent.
Strict (tts_strict)	Off	On + a pinned model means **only** that model is used, no fallback. Voice stays identical, but the video may stop when quota runs out.
Cloud voices	(empty)	Comma list of Cloud TTS voices. Empty = per-language default (en: en-US-Neural2-D, en-US-Standard-D · ur: ur-IN-Wavenet-A, ur-IN-Standard-A).

The hint shown next to a TTS model means:

Model kind	Hint
...-lite-tts	Lite: lighter, bigger free daily allowance.
...-preview	Preview build: can change or disappear.
gemini-2.5-...	Older model: safe fallback.
Others (standard flash TTS)	Tried first, but smaller free daily allowance.

> **Heads up**
> About Urdu: Urdu is **not** in Google's official Gemini-TTS language list. Gemini detects the language itself and often speaks it, but always run a short test first. Cloud TTS (ur-IN voices) is the fallback.

4.4 Default thumbnail template

The project's thumbnail settings (set in the Thumbnail studio, section 11): template image, font **Fredoka**, size **110**, colour **#ffffff**, stroke **#2a2550**, align **left**, position x **6**, y **50**, max width **58**, and **shade** (on).

5. The Videos tab: daily work

Opening a playlist (or Singles) shows the **Videos** tab. From here you add videos, change their settings, upload sheets and watch them get built.

5.1 The stats line at the top

The top line looks like: **12 videos · 38.5 minutes · 3 hold · 5 waiting · 4 completed**. It shows the number of videos, total minutes (a video with empty Minutes counts as the project default) and a count per status (only those above zero).

If some videos have no thumbnail of their own, a note appears: *“N videos have no thumbnail of its own”*, saying that the default template will be used (or a plain background if there is no template). The **Set the default template** link inside the note takes you straight to the Thumbnail studio.

5.2 Adding a new video

Above the list there are two tabs: **From a prompt** and **I have a script**.

Tab 1: From a prompt (Frame writes the script)

Field	Notes
Video title	The video's name. The mp4 file takes this name. If empty, the first 60 characters of the topic become the title.
Topic or prompt for the pipeline	The topic the script is written about. This is exactly what the backend receives as “topic”. If empty, the title is used as the topic.
Minutes (small number box)	The placeholder shows the project's default length (e.g. **3 min**). Leave empty to use it. Minimum 0.5, step 0.5.
Add video	Saves the row to the sheet. At least a title or a topic is required.

Tab 2: I have a script

Field / button	Notes
Video title	If empty and you uploaded a file, the **file name** (without extension) becomes the title; otherwise the first 60 characters of the script.
Minutes	Placeholder **auto**: leave empty and the length comes **from the script**.
Script text box	Paste the full script. A character counter is shown below.
Upload script file	Formats: **.txt, .md, .srt, .pdf, .docx, .doc, .rtf, .odt**. The file's text goes into the box above (replacing the old text).
Add video	Disabled while the script is empty.

> **Heads up**
> The script limit is **49,000 characters** (one Google Sheet cell holds about 50,000). A longer script gives the error *“split it into two videos”*. Split long scripts into two videos.

> **Mind the status**
> A video added from this tab gets status **Waiting** straight away, so the backend may build it on its next run. If you want to review first, open the video card after adding and set it to **Hold** (or change Status in the sheet).

5.3 Sheet upload: many videos at once

Upload a sheet brings many videos in from an Excel/CSV file. Formats: **.xlsx, .xls, .csv, .tsv, .ods**. Each row = one video, each column = one setting of that video (column list in section 7).

1. Press **Upload a sheet** and choose the file.
2. Frame checks the columns and tells you: **how many videos were found**; **Matched columns** (recognised); **New columns, added to your sheet** (not in your sheet yet, they will be added); **Not in your file, left empty** (defaults will apply).
3. If none of **Video Title**, **Topic / Prompt** or **Script** exists, a red error appears and the buttons stay disabled. Rename one column and upload again.
4. Then two choices: **Append to current sheet** (old videos stay, new ones are added below, numbering continues) or **Replace current sheet** (asks for confirmation; all current videos and their statuses are removed). **Cancel** goes back.

Frame understands many alternative column names, e.g. **Title / Name / Video name** = Video Title, **Prompt / Topic** = Topic / Prompt, **Length / Duration** = Minutes, **Desc** = Description, **Ratio / Shape** = Aspect, **Renderer / Library** = Engine, **Research / Search** = Web Research, **Overlays** = Custom Clips, **Instruction(s)** = Custom Instruction, **Source text / Book text / Voiceover** = Script.

> **Check status after upload**
> The on-screen note says *“Status becomes Waiting when empty”*, but the import code in config.js turns an empty status into **Hold** (and an upload can only give Hold or Waiting). So **always check the Status column after an upload** and set the videos you want built to Waiting.

If the file has a **Script** column filled but Input Type empty, the video is treated as “Script”; if **Source Mode** is filled as well, as “Source”. If there is no title, the first 60 characters of the topic or script become the title.

5.4 The video card: every detail

Each video is a card. The closed card shows one line: **number · title · “own script” or “prompt” · minutes · aspect (own/project) · thumbnail (own/default) · description state**, and the status pill on the right. Click the card to open it.

Card fields

Field	What it does
Title	The video's name; also the output file name.
This video is made from	**A prompt** (Frame writes the script) or **My own script** (used as written). (The Source type is set through sheet columns, section 6.3.)
Script / Topic	In Script mode the full script (with character counter and **Upload script file**). In Prompt mode the **Topic / prompt**: “this is exactly what the pipeline receives as the topic”.
Description	This video's own description. It is written first, then the playlist's default description is appended. Singles use only this one. (Record only; no effect on how the video is made.)
Minutes	Empty = playlist default (or from the script).
Audience	Empty = playlist audience.
Narration language	Playlist default / English / Urdu.
Screen language	Playlist default / English / Urdu.
Aspect	Project default / 16:9 / 9:16 / 1:1 / 4:5. A hint below says how text and layout will look in that shape.
Voice	Playlist narrator / any of the 30 voices, each shown with its tone (e.g. *Kore · Firm*).
Extra config (JSON)	Any VideoConfig field, e.g. **{"scene_sec": 10, "quality": "high"}**. **Stronger than every other level** (section 15).
Notes	Your notes. The backend writes here too (progress, errors, scene summary).

Card buttons

Button	What it does
Save video	Saves your changes to the sheet. In Script mode an empty script gives an error (“set to Script, but the script is empty”); so does a script over 49,000 characters.
Upload own thumbnail / Replace own thumbnail	Your own image for this video. While it exists it is used instead of the default template.
Use default thumbnail	(Shown only when an own thumbnail exists.) Removes the own thumbnail and goes back to the default template.
Thumbnail studio	Opens the Thumbnail studio with this video's row (section 11).
Set back to Waiting	(Shown when the status is not Waiting.) Sets the video to **Waiting** again: release a Hold, or rebuild a Failed/Completed video.
Output	(Shown after it is built.) Link to the finished video in Drive.
Captions (.srt)	(If captions exist.) Link to the captions file.
Remove	Asks for confirmation; the row is **deleted** from the sheet. Files in Drive **stay**.

A line at the bottom of the card says which thumbnail will be used: *own thumbnail* (with a link) or *default template of this project*.

5.5 The three buttons above the list

Button	What it does
Download jobs for backend	Downloads **frame_jobs.json**, containing the fully merged config of only the **Waiting** videos (for a manual/local backend). If no video is Waiting you get the error “No video is Waiting.” If a row's Extra Config is not valid JSON you also get an error naming the row.
Import results	Brings a results **.json** file made by the backend back into the sheet (status / output link). If no row in the file matches this playlist: “No row in that file matches this playlist.”
Open sheet	Opens this playlist's Google Sheet in a new tab. You can type or edit rows there directly.

> **Tip**
> With the normal GitHub Actions setup you do **not** need Download jobs or Import results: the backend reads Drive and updates the sheet itself. These buttons are for people who run the backend manually on their own machine.

6. Input types: Prompt, Script, Source

6.1 Prompt: Frame writes the script

You give a topic. The backend (1) does web research (if on), (2) writes the script, taking audience, minutes, language and the optional **Custom Instruction** (up to 2,000 characters) into account, (3) runs the rest of the pipeline. When the video is longer than 4 minutes the script is planned in chapters (**Chapter length**).

6.2 Script: your script exactly as written

The script's narration is used **word for word**. Frame only:

- **Cleans** it: markdown (bold, italics, links, bullets, rules), SRT time-codes and numbers, HTML tags.
- Turns a markdown **# heading** into the scene **title** (the heading is not spoken). With no headings, one cheap Gemini call writes a short heading per scene.
- **Script cleanup** (default on): Gemini may remove notes, metadata and stage directions and fix punctuation, but **may not change a word**. The code compares every paragraph with the original and discards any paragraph whose wording changed.
- Splits it on sentence boundaries into scenes of about **Seconds per scene** (default 15 s).
- The rest of the pipeline (voice, visuals, rendering) runs as usual.

Leave **Minutes** empty and the length is taken from the script. If the row's **Language** is empty and the script is in Urdu script (Arabic letters), the language automatically becomes **ur**; if it is Latin, **en**.

> **Heads up**
> If Input Type = Script but the Script cell is empty, the frontend's jobs file treats the row as a prompt, but the GitHub backend fails with “Input Type is Script but the Script cell is empty”. So always fill in the script in Script mode.

6.3 Source: adapting raw text

With Source you give the text of a book/article/notes and say how much Frame may change it. The text goes in the sheet's **Script** column; set **Input Type = Source** and choose a **Source Mode**. (The Videos tab dropdown has only Prompt and Script; Source is set through sheet columns or upload.)

Source Mode	What it does	When to choose it
polish (Light polish)	Keeps the book's **own words**. Only cleans it for speaking: removes page numbers, footnotes and headings, fixes typos, splits very long sentences. Almost no change.	The text is already good and just needs to be speakable.
rewrite	Same facts, same order, **new wording**. Written to be heard, in the playlist's narrator style and audience level.	The text is bookish/difficult but the order must stay.
reference	Frame reads the text and builds **its own lesson** from it: it may reorder, shorten, simplify and add examples, but **must not add facts the text does not support**.	The text is only a reference and you want a fresh kind of lesson.
custom	Only the **Custom Instruction** you write is the whole brief.	You want a specific approach of your own. This mode can run on the instruction alone, without text.

Custom Instruction in Source is an extra instruction on top of the mode (max 2,000 characters). It is optional for Prompt videos too. Minutes: for polish/rewrite the length comes from the text; for reference/custom it is Minutes or the project default.

7. The playlist sheet columns

In each playlist's **Playlist** tab every video is one row. You edit through the frontend, or directly in the sheet if you prefer. Column names are not case sensitive. A column missing from the sheet is ignored by the backend; columns the backend writes itself (like Captions Link) are added automatically.

Column	Filled by	Notes
#	Frontend	The video's number (order).
Video Title	You	Name; output file **<title>.mp4**. If Topic is empty the title is the topic.
Topic / Prompt	You	Topic of prompt videos.
Input Type	You	**Prompt** / **Script** / **Source** (empty = Prompt; Script column filled = Script).
Script	You	Script or Source text (max 49,000 characters).
Source Mode	You	polish / rewrite / reference / custom (Source only).
Custom Instruction	You	Extra instruction (max 2,000 characters).
Minutes	You	Empty = playlist default or from the script.
Status	You + Backend	Hold / Waiting by you; the rest by the backend (section 2.3).
Description	You	This video's description (the playlist default is appended after it). **The backend does not read it when making the video**; it is for the record.
Audience · Language · Screen Language · Aspect · Voice	You	Overrides for this video. Empty = playlist/project value.
Engine · Engine Fixed · Web Research	You	Engine for this video (auto/manim/remotion/both), keep the engine fixed (1/true/yes/y/fixed), and web research (0/false/no/n/off turns it off).
Thumbnail Text	You	Text written on the thumbnail. Empty = the title. (The backend does not read it; thumbnails are the frontend's job.)
Thumbnail Link	Frontend	Your own thumbnail (the link is filled in when you upload). The backend does not touch it.
Custom Clips (JSON)	Frontend	Images/clips overlaid on this video (section 10).
Extra Config (JSON)	You	Any VideoConfig field. Highest priority (section 15).
Scenes (JSON)	Frontend / You	The scene list (section 9). Plain text works too: **Scene 1: ...**.
Output Link	Backend	Drive link of the finished video.
Captions Link	Backend	Link of the captions (.srt); the backend adds the column if it is missing.
Video Folder ID · Input Folder ID · Output Folder ID · Video Folder Link	Frontend	This video's own Drive folder. The backend reads input from there and writes output to **Output Folder ID**. Do not edit these.
Notes	You + Backend	Your notes; the backend writes progress / errors / scene summary here.

Column	Filled by	Notes
Updated	Frontend/Backend	Time of the last change.

The sheet also has a **Log** tab where the backend writes three kinds of entries per video: **Build started**, **Video saved** or **Video failed** (with time).

8. The look of the video: engine, shape, quality, scenes

8.1 Video engine: Manim or Remotion

Each scene is made with one of two tools. The **Video engine** setting decides which:

Option	Name on screen	Best for
auto	Let AI choose	The AI reads the script and picks the better engine **per scene**; it may use one or both. Default. Remotion is normally preferred; Manim only for real maths.
manim	Manim only	Maths, diagrams, graphs, step-by-step explanations. **Slower** to render, very precise.
remotion	Remotion only	Text, cards, charts, flows, timelines, counters, branded motion graphics, Urdu text. **Faster** to render.
both	Both (mix)	Manim for technical scenes, Remotion for the rest. For the backend this is **the same as auto** (every scene goes to one of the two).

Fallback: if the chosen engine cannot render a scene even after the repair attempts, the other engine is tried, then a plain title card. One scene never stops the whole video. **Keep my engine choice fixed** / **Engine Fixed** turns this fallback off, and then the video fails.

Remotion needs Node.js 18+ on the machine (already present in the GitHub backend). Without Node every scene is made with Manim. Each scene's actual engine is recorded in the report saved with the video.

8.2 Aspect ratio (shape of the video)

Aspect	Final size	For	Layout
16:9	1280 x 720	YouTube, normal video	Normal text size, layout side by side.
9:16	720 x 1280	Shorts, Reels, TikTok	Bigger text, layout stacked top to bottom.
1:1	720 x 720	Square posts	Slightly bigger text, compact layout.
4:5	720 x 900	Portrait feed posts	Vertical-style behaviour.

Priority: **video row > project > 16:9**. The words **9x16, 9/16, vertical, shorts, reels, tiktok, square, wide, landscape, youtube** are also understood. In a sheet row the word **portrait** means **9:16** (write “4:5” for 4:5). An invalid value falls back to the playlist's value (or 16:9). A wide and a vertical version of the same topic are built in separate folders and never overwrite each other.

8.3 Render quality

Quality	Short side of render	Speed
Low	480 px	Fastest, for tests.
Medium (default)	720 px	Normal use.
High	1080 px	Sharpest, slowest (especially with Manim).

The short side of the final video is always 720 px (see the table above); quality only sets how the clips that are joined into it are rendered.

8.4 Scene pace: seconds per scene and scene count

Number of scenes = **video length / Seconds per scene**, never above **Max scenes** (default 60). Example: 3 minutes at 15 s per scene = 12 scenes. A bigger Seconds per scene = fewer scenes = fewer renders = faster; smaller = more movement but slower.

The frontend also computes a **render estimate** (an estimate, not a promise): per scene Manim low/medium/high = 6/14/40 s, Remotion = 4/8/20 s, plus about 60 s per video and 3 s per scene of overhead (script, voice, joining). Example: 3 min, 15 s/scene, medium, auto = about **3 to 4 minutes**.

8.5 Language and Urdu notes

- Narration and screen language can differ (e.g. Urdu voice, English screen text).
- For Urdu screen text the render machine needs an Urdu font (default **Noto Naskh Arabic**), otherwise text shows as boxes. The GitHub backend already installs it.
- **LaTeX turns itself off** with Urdu screen text.
- For Urdu voice see the Gemini TTS note in section 4.3; Cloud TTS is available as a fallback.

9. Scenes: your own storyboard or code file

For each video you can give a list of scenes. Each scene is either a **description** (Frame builds the animation) or **your own file**, which runs **exactly as it is**. The list is saved in the sheet's **Scenes (JSON)** column. (The Scenes screen code was not in the zip; the details below come from the backend and README.)

9.1 Two kinds of scene

Kind	What you give	What happens
Description	A title (optional) and description (up to 4,000 characters).	Frame builds the script/animation from it; engine auto or the one you pick.
Own file (Manim)	A **.py** file	The first Scene class that has **construct()** is rendered.
Own file (Remotion)	**.tsx .jsx .ts .js .mjs** or **.zip**	A component file is wrapped to the video's size and the narration length. A file/zip with **registerRoot + <Composition>** runs at its own size/length. A zip with **package.json** gets its own npm install.

The engine is guessed from the file extension (.py = Manim, others = Remotion). The file must be under **60 MB**. Frame **never edits** your file; it only runs it, in a separate process, without API keys/tokens, with a time limit.

9.2 Writing scenes as plain text

You can type plain text in the Scenes (JSON) cell; it is recognised in this order:

- Headers: **Scene 1: title**, **Scene 2 - title**, **Shot 3**, **Sequence**, or the Urdu-script word for “scene” followed by a number (the lines after it are the description).
- If there are no headers, numbered lines like **1. text** / **2) text**.
- Otherwise blocks **separated by blank lines**; otherwise each line is one scene.

```
Scene 1: Hook
Start with a small story: a car on a road whose speed keeps changing.

Scene 2: The velocity graph
Show time on the x-axis and position on the y-axis, and highlight the slope.
```

A video can hold at most **200 scenes**.

9.3 Scene count: Frame handles it for you

A video can only carry a sensible number of scenes for its length (too few and every scene drags, too many and the screen flickers). Frame first works out the budget:

Situation	What happens
Your scene pace is within 1.5x faster or 1.5x slower than scene_sec	The list is kept **exactly as it is**.
Too few scenes	Frame creates new scenes itself: hook, bridge/analogy, worked example, recap. Your scenes stay where they are. (In a Script video these are “free” parts Frame designs from the narration.)

Situation	What happens
Too many scenes	The least important scenes are dropped (first and last scene, with own-file scenes dropped last of all). A dropped scene's description is joined to its neighbour as **“also cover”**, so no idea is lost.

Setting (Extra Config / project config)	Default	Meaning
scene_fit	auto	**auto** = the logic above. **exact** = your list **exactly as given** (cut only by the hard limit, never new scenes added).
scene_min_sec	6	A scene shorter than this is never planned (minimum 2).

9.4 Which scene goes where

- **Prompt video:** the script is written around your storyboard: one scene = one script scene, in your order; chapters take consecutive runs of scenes.
- **Script / Source video:** the narration is cut by sentences and Gemini assigns every scene to the part it illustrates (looking at the title, description and on-screen text of own files). If the answer is wrong or Gemini is down, scenes are spread evenly over the narration.
- **Own-file scenes:** their narration is written by looking at the on-screen text of that file.

9.5 If your own file cannot run

If the file cannot be downloaded or fails to render, that scene is built **from its description** (or a title card) instead; the video does not fail and the report says so. If your clip has a different shape from the video it is not cropped: it is shown **complete over a blurred background**. If it is shorter than the narration, its **last frame is held**.

9.6 Scene report

A **<title>.scenes.json** file is saved next to the video in the Output folder: for each scene kept / added / dropped / folded, and which engine drew it. A line appears in the sheet's **Notes**, for example: *“Scenes: 12 scenes from your 5, 2 your own file, 7 added by Frame”*.

10. Custom Clips: your own image/video overlays

Custom Clips are your own images or video clips meant to be overlaid on **one video only** (a logo, a face, a product shot). They are never inherited from the project or global level. The files are kept in that video's Drive folder under **Input/clips/**. Up to **20 clips** per video.

Option	Default	Range / values	Meaning
File	-	image/* or video/*	An image or a video. The kind is detected automatically.
Start (sec)	0	0 to 36000	From which second of the final video it appears.
Duration (sec)	5	0.5 to 36000	How long it is shown.
Position	bottom-right	top-left · top-right · bottom-left · bottom-right · center · full	**full** covers the whole frame.
Width (%)	25	5 to 100	Share of the frame width.
Opacity (%)	100	5 to 100	How visible (100 = fully opaque).
Loop	Off	on/off	Video clips only: repeat the clip until the duration is filled.
Mute	On	on/off	Video clips only: silence the clip's own sound.
Note	-	200 characters	A reminder for yourself.

Clips: one important point
In this zip's backend the clip data (**custom_clips**) is read from Drive and reaches the config, but **I could not find the code that actually overlays it during rendering/assembly**. Clips may therefore be saved yet not appear in the video. Test with a short video first, and if they do not appear the backend needs an overlay step added.

11. Thumbnails

Thumbnails are the frontend's job (Thumbnail studio). **The GitHub backend does not touch thumbnails** and makes none with the video. Which thumbnail is chosen:

Situation	Thumbnail used
The video has its own Thumbnail Link	That one (the image you uploaded).
No own thumbnail, but the project has a default template	The template, with the video's **Thumbnail Text** (or title if empty) written on it.
Neither	A plain background with the title written on it.

Thumbnail studio settings (project level)

Setting	Default	Meaning
Template	(none)	Background image the title is written on. For all videos of the project.
Font	Fredoka	Font of the title.
Size	110	Font size.
Color	#ffffff	Text colour.
Stroke	#2a2550	Outline colour around the text (for readability).
Align	left	Text alignment.
X / Y	6 / 50	Text position (percent of the frame).
Max width	58	Maximum text width (%), then the line wraps.
Shade	On	A soft shadow behind the text so it stays readable.

From the video card: **Upload own thumbnail** / **Replace own thumbnail** / **Use default thumbnail** / **Thumbnail studio** (section 5.4). (The studio screen code was not in the zip.)

12. Singles: stand-alone videos

If you want a video or two without creating a playlist, use the **Singles** tab. (The Singles screen code was not in the zip; behaviour comes from the README and backend.) All Videos-tab features (prompt/script, thumbnails, sheet upload, card buttons) work the same here, except:

Item	In Singles
Folder	**Frame AI/Singles/**
Sheet	Its own sheet: **Singles - Record**
Narrator / thumbnail style	Singles has its own; nothing is taken from a playlist.
Description	**No shared description.** Only the video's own description is used (if any). The placeholder when adding says so.
Output	**Singles/Output/**
Production plan	If Singles (or any new playlist) is not yet saved in the plan, the backend adds it **at the end of the plan by itself**. A paused project is still skipped.

13. Production tab: whose turn is it

When there are several playlists with **Waiting** videos, the Production plan decides whose turn comes first. (The screen code was not in the zip; options come from config.js.) The plan is saved in the **Production Plan** sheet in Drive.

13.1 Plan options

Option	Default	Meaning
Order: rotate	(default)	**A, B, C, A, B, C ...**: each round takes “videos per turn” videos from each playlist.
Order: sequential	-	**All of A first, then all of B ...**
State: active / paused	active	A **paused** playlist stays out of the queue; its videos are not built.
Videos per turn	1	How many of this playlist's videos per turn. In the GitHub backend run, **0 or empty = all Waiting videos**.
Pause after (min)	0	How many minutes to wait after this playlist's video before starting the next (0 = none).
Pause every (videos)	1	After how many videos that pause applies (e.g. every 3 videos).

A newly created playlist is added to the **end of the plan, active**; a deleted playlist leaves the plan.

13.2 What the plan produces

- **The queue:** the Waiting videos in the right order. Each queue entry has its position, playlist, row and pause time.
- **production_record.json:** an at-a-glance record: the next video (*next*), *on_hold*, *in_process*, *completed*, *failed*, the full *queue*, *paused_projects* and a count per status (*summary*).
- **One jobs file for all playlists**, already in build order (for a manual backend).

> **Backend and the plan**
> The GitHub Actions backend reads only three things from the plan sheet: **order** (the order of playlists, a number), **state** (active/paused) and **videos per turn**. The rotate/sequential and pause options matter only for the frontend queue and jobs file. The GitHub backend goes through the playlists in plan order on every run and builds from each as many videos as “videos per turn” says (0 = all). So in one run A, B and C each build their videos per turn, and again on the next run.

14. Backend behaviour: what you will see

14.1 When it runs

- **Every 6 hours** by itself (00:00, 06:00, 12:00, 18:00 UTC).
- Or immediately from GitHub: **Actions > “Frame AI - build waiting videos” > Run workflow**.
- **Only one run at a time.** A second run waits for the first so that they do not fight over the same rows.
- A run does not start any new video after **300 minutes** (GitHub's limit is 360); the rest wait for the next run.

14.2 What happens to one video

Stage	What happens
Status changes	The row becomes **In Process**, Notes shows **[gh-action] started <time>**, and the Log tab gets “Build started”.
Config merge	Global < project < row < Extra Config are merged into this video's config (section 2.2).
0. Research	(If Web Research is on) one Google search; notes are saved. **If it fails the video is Failed.**
1. Script	Prompt: Gemini writes it · Script: cleaned and split into scenes · Source: adapted by the mode. If a scene list was given, the storyboard plan is also made here.
2. Voice	Each scene's narration through TTS; the real audio gives each sentence's timing (for captions and sync).
3. Analysis	Concepts, examples, key points, visuals.
4. Spec and engine	Animation steps are pinned to sentences; each scene is assigned Remotion or Manim.
5. Code + render	Each scene's code is written and rendered; on a problem: auto-repair (default 2 times), then the other engine, then a title card.
6. Sync + assemble	Each scene is matched to its voice and everything is joined. The final MP4, **.srt** captions and a report are produced.
Save to Drive	**<Video Title>.mp4** in the Output folder; captions **<Video Title>.srt**; if scenes were given, **<Video Title>.scenes.json**. Status becomes **Completed** and the Output Link and Captions Link are filled in.

> **Heads up**
> Characters like **\ / : * ? " < > |** in the title are replaced by spaces and the name is cut to 120 characters. **If a file with the same name is already in the Output folder, the new file replaces it** (no duplicate). Do not give two videos the same title or they will overwrite each other.

14.3 Outcomes: Completed, Failed or back to Waiting

Outcome	What the sheet shows	What you do
Completed	Status Completed, Output Link, Captions Link, and a scene summary in Notes (if scenes were given).	Watch the video. To rebuild, use **Set back to Waiting**.
Failed	Status Failed, Notes shows **[gh-action] <reason>** (up to 300 characters). The Log tab shows “Video failed”.	Read the reason (section 16), fix it, then **Set back to Waiting**.
Waiting (quota)	Notes: **[gh-action] quota, retry next run**. The run stops there.	Nothing. The next run reuses the cached script and voice and moves on quickly.

One playlist's problem does not stop the others: if a playlist hits a serious error the backend skips it and goes to the next. But when **quota runs out** the whole run stops (quota is shared by everything).

14.4 Free-tier considerations

- On the Gemini free tier every model has a **daily limit**. A long chain (section 3.2) and several keys from different projects raise it.
- With many waiting videos, use **Videos per turn** to keep each run small so quota is not used up at once.
- GitHub free private repos get roughly **2000 minutes** a month; runs every 6 hours and long videos can use that up.

15. Extra Config (JSON): every option

In a row's **Extra Config (JSON)** column you can write **any** of the fields below. It has the highest priority. The format is a JSON object:

```
{"scene_sec": 10, "quality": "high"}
```

- If the JSON is invalid the backend **silently ignores it** (only a log line “bad Extra Config JSON, ignored”) and builds from the other settings. The frontend's “Download jobs” stops with an error instead. So validate the JSON after writing it.
- Names not in the list below are ignored by the backend. Nested objects are flattened.
- Text values go in double quotes; true/false without quotes.

Field	Default	Values / meaning
minutes	3	Length of the video (minutes).
audience	curious beginners	Script level.
language	en	en | ur (narration).
screen_language	(= language)	en | ur (screen text).
input_type	auto	prompt | script | source.
script / source_text	-	The script or Source text.
source_mode	polish	polish | rewrite | reference | custom.
custom_instruction	-	Extra instruction (up to 2,000 characters).
script_cleanup	true	Own script only: Gemini removes notes/metadata and fixes punctuation (never changes words).
voice	Charon (backend) / Kore (project)	One of the 30 voices (Appendix A).
tts_style	calm, clear and friendly...	Delivery style (free sentence).
tts_engine	auto	auto | gemini | cloud.
tts_model / tts_strict	- / false	Pinned TTS model; strict = only that one.
cloud_voices	-	Comma list of Cloud TTS voices.
light_models / medium_models / code_models / tts_models	global	Comma list. Change model order for this video only.
rpm / tts_rpm	5 / 4	Requests per minute.
max_wait	240	Longest quota wait (seconds).
tts_group_scenes / tts_group_chars	4 / 4000	Scenes / characters per TTS call.
scene_sec	15	Approximate seconds per scene.
chapter_min	2	Chapter length (videos over 4 min).
scene_batch	4	Scenes' narration written per Gemini call.
max_scenes	60	Upper limit of scenes.
min_steps_per_sentence	2	Minimum animation steps per sentence.

Field	Default	Values / meaning
scene_fit	auto	auto | exact (section 9.3).
scene_min_sec	6	Shortest scene (minimum 2).
quality	medium	low | medium | high.
aspect	16:9	16:9 | 9:16 | 1:1 | 4:5.
engine	auto	auto | remotion | manim | both.
engine_fixed	false	true = no fallback to the other engine.
retries	2	Auto-repair attempts per scene.
allow_latex	false	LaTeX in Manim (automatically false with Urdu screen text).
web_research	true	false = write without research.
urdu_font	Noto Naskh Arabic	Font for Urdu screen text.

> **Tip**
> **workdir** and **output_dir** always stay **build** / **output** on the GitHub backend and cannot be changed through Extra Config. **Custom Clips** and **Scenes** have their own columns (sections 10 and 9); do not put them in Extra Config.

Examples

```
{"scene_sec": 8, "max_scenes": 40}                          // faster-changing video
{"engine": "remotion", "engine_fixed": true}                // Remotion only, no fallback
{"language": "ur", "screen_language": "en", "voice": "Charon"}
{"scene_fit": "exact", "scene_min_sec": 8}                  // my scene list exactly as written
{"web_research": false, "quality": "low"}                   // quick test video
```

Note: the parts after // above are only explanations. Real JSON has no comments; remove them before pasting.

16. Common problems and fixes

Problem / message	Cause	What to do
Video stays on Hold	The backend never touches Hold.	In the video card use **Set back to Waiting** (or set Status = Waiting in the sheet).
It is Waiting but not built	The run has not happened yet (every 6 hours); or the playlist is **paused** in the plan; or the "videos per turn" limit; or an earlier video stopped on quota.	Wait for the next run or use **Run workflow** on GitHub. Check state and videos per turn in the Production tab.
Failed: web research message	Research is on and the Google search failed. The video is deliberately not written without research.	Set it Waiting again. If it keeps happening, turn **Web Research** off (0/false in the row, or the project checkbox).
Failed: "Input Type is Script but the Script cell is empty"	Script mode but no script.	Fill the script or set Input Type to Prompt.
Failed: "Source Mode is custom but the Custom Instruction cell is empty"	Custom mode with no instruction written.	Fill the Custom Instruction.
Failed: "language must be one of ['en', 'ur']" or an aspect/quality message	A wrong value in the row or Extra Config.	Use a valid value from section 4 or 15.
Notes: "quota, retry next run"	Gemini/TTS daily or per-minute limit.	Nothing; it resumes on the next run. More keys (from different projects) or fewer videos per turn help.
Urdu text shows as boxes	The Urdu font is missing on the render machine.	Fix the font name in the Control room or keep the screen language English.
Voice sounds slightly different between videos	The model switches when quota is hit.	**Pin a TTS model** in the narrator; for full consistency also turn on **Strict**.
Sheet upload buttons are disabled	No Video Title, Topic/Prompt or Script column found.	Rename one column to one of these and upload again.
Videos are on Hold after upload	The code turns an empty Status into Hold.	Set the ones you want built to Waiting (section 5.3).
Extra Config had no effect	If the JSON is invalid the backend silently drops it.	Validate the JSON: straight double quotes and no trailing comma.
A Custom Clip does not show in the video	No overlay step was found in this backend.	See the "Important" note in section 10.
A new video erased an older file	Same title = same file name = replaced.	Give every video a different title.
A video takes very long	Many scenes, High quality, or Manim.	Raise Seconds per scene, use Low/Medium quality, use the Remotion engine, or split a long video in two.
"The script is N characters..."	The script is over 49,000 characters.	Split it into two videos.

Problem / message	Cause	What to do
“No video is Waiting.”	Download jobs found no Waiting video.	Set at least one video to Waiting first.

17. Quick recipes: by task

17.1 Your first video from a prompt

1. Open the playlist, go to the **Videos** tab and choose **From a prompt**.
2. Enter a title and topic, and optionally minutes (otherwise the default).
3. Press **Add video**. The video is **Waiting**.
4. Wait for the next run (or use Run workflow on GitHub). Status goes **In Process** then **Completed**.
5. Use the card's **Output** button to watch it and **Captions (.srt)** for the captions.

17.2 A video from your own script

1. Choose the **I have a script** tab.
2. Paste the script or press **Upload script file** (.txt, .md, .srt, .pdf, .docx, .doc, .rtf, .odt).
3. Leave minutes empty (auto). If you like, write **# Heading** lines in the script: they become scene titles.
4. Press **Add video**. The narration will be your script's exact words.

17.3 A video from a book or article (Source)

1. Make an Excel/CSV file with columns **Video Title**, **Input Type** (= Source), **Script** (the book text), **Source Mode** (polish / rewrite / reference / custom), and optionally **Custom Instruction**.
2. Use **Upload a sheet** and choose **Append**.
3. Check the statuses and set the ready videos to **Waiting**.

17.4 Many videos at once

1. Prepare a sheet file: one row per video; use the column names from section 7.
2. **Upload a sheet** > check **Matched columns** / **New columns** > **Append** (or **Replace**).
3. Set **videos per turn** in the Production tab so that they are not all built in one run.
4. Set the ready videos to Waiting.

17.5 Making a vertical Short / Reel

1. Set **Default video shape** = **9:16** in the project (or only Aspect = 9:16 in one video's card).
2. Keep minutes short (0.5 to 1) and **Seconds per scene** small (e.g. 8).
3. Add the video and set it to Waiting. The final size is 720 x 1280.

17.6 Urdu narration with English screen text

1. Set Narration language = **Urdu** and On-screen text language = **English** (in the project or the video card).
2. Choose a voice (make a test video first; see the Urdu TTS note in section 4.3).
3. If you need Urdu on screen, check the render machine's Urdu font.

17.7 Rebuilding a video

1. Open the video card, change what you need (script, topic, settings) and press **Save video**.
2. Press **Set back to Waiting**.

3. If the title is unchanged, the new video replaces the old file.

17.8 Using your own scene (Manim or Remotion file)

1. In the Scenes section add a scene and choose the **own file** option; the file is uploaded to Drive.
2. Keep the extension correct (.py Manim; .tsx/.jsx/.ts/.js/.mjs/.zip Remotion) and the size under 60 MB.
3. If you want, write **scene_fit = exact** so that Frame does not add scenes to your list.
4. After the video is built, check **<title>.scenes.json** to see whether your file was used or fell back to the description.

17.9 Keeping the voice consistent

1. Choose a **Voice** in the narrator (e.g. Charon or Kore).
2. In the same place **pin a TTS model**.
3. For complete consistency turn **Strict** on (but a video may stop when quota runs out).

Appendix A: Gemini voices

In the frontend each voice shows its tone (the Voice dropdown shows “Name · Tone”). The tone and feel below follow the on-screen hints.

Voice	Tone	How it sits in a lesson
Zephyr	Bright	light and cheerful opener
Puck	Upbeat	lively, keeps kids awake
Charon	Informative	clear explainer, steady pace
Kore	Firm	confident teacher
Fenrir	Excitable	big-reaction hype
Leda	Youthful	young, fresh classroom voice
Orus	Firm	solid and authoritative
Aoede	Breezy	relaxed and flowing
Callirrhoe	Easy-going	casual chat feel
Autonoe	Bright	clean and optimistic
Enceladus	Breathy	soft, close-to-the-mic
Iapetus	Clear	plain and precise
Umbriel	Easy-going	calm and unhurried
Algieba	Smooth	polished, even flow
Despina	Smooth	mellow and rounded
Erinome	Clear	crisp diction
Algenib	Gravelly	textured, character voice
Rasalgethi	Informative	measured lecturer
Laomedeia	Upbeat	positive and moving
Achernar	Soft	gentle, good for slow topics
Alnilam	Firm	direct and decisive
Schedar	Even	level, no surprises
Gacrux	Mature	seasoned and grounded
Pulcherrima	Forward	upfront and assertive
Achird	Friendly	approachable guide
Zubenelgenubi	Casual	everyday conversation
Vindemiatrix	Gentle	kind, patient
Sadachbia	Lively	animated storyteller
Sadaltager	Knowledgeable	expert tone
Sulafat	Warm	warm and reassuring

Appendix B: Short glossary

Term	Meaning
Playlist / Project	A group of videos with its own sheet, folder and settings.
Singles	A separate folder and sheet for stand-alone videos.
Row	One line of the playlist sheet = one video.
Scene	A short part of the video (about 15 seconds) with its own narration and animation.
Narration	The text of the spoken voice in the video.
TTS	Text-to-speech: making a voice from text.
Manim	A Python tool for maths/diagram animations.
Remotion	A React-based tool for motion graphics and cards.
Aspect ratio	The shape of the video: 16:9, 9:16, 1:1, 4:5.
Quota / rate limit	The free-tier limit of Google (how many requests in how much time).
Fallback	Trying a second way when the first fails.
Overlay	Placing an image/clip on top of the video.
Captions (.srt)	The subtitle file that goes with the video.
Pin (TTS model)	Making one model the permanent first choice.

> **A last word**
> Everything in this guide follows the files in the zip (config.js, Videos.jsx, the backend Python files and the README). For screens whose code was not in the zip (Control room, Singles, Production, Thumbnail studio, Scenes) the real button names or placement may differ; the list of options and their meaning will stay the same.